

AGENTIC AI

Agentic Workflows

Chapter 1: Foundations, Architecture, Patterns & Implementation

A Comprehensive Technical Reference for Production-Grade GenAI Systems

Target Audience: Intermediate Practitioners | Style: Academic & Formal

SECTION I: THEORETICAL FOUNDATIONS

1. Introduction to Agentic Workflows

The field of artificial intelligence has undergone a profound paradigm shift. Whereas earlier AI systems were largely passive — responding to isolated queries without context or continuity — the contemporary landscape is increasingly defined by AI agents: autonomous systems capable of perceiving their environment, forming plans, executing multi-step actions, and adapting dynamically to new information. This chapter provides a rigorous introduction to agentic workflows, establishing the theoretical foundations necessary for designing and implementing production-grade AI systems.

1.1 What Is an AI Agent?

At its most fundamental level, a modern AI agent is a computational system that employs a Large Language Model (LLM) as its central processing unit — its 'cognitive engine' or 'brain'. However, an agent is far more than a wrapper around an LLM API call. A true agent is an architecturally complete system engineered to:

- Perceive its environment by ingesting structured and unstructured data from diverse sources.
- Reason about observations by applying logical inference, contextual understanding, and domain knowledge.
- Formulate plans by decomposing high-level goals into actionable sub-tasks.
- Execute actions by invoking external tools, APIs, databases, or other agents.

This cyclical loop — perception, reasoning, planning, and execution — is commonly referred to in the literature as the Perceive-Reason-Plan-Act (PRPA) cycle. It is this capacity for closed-loop, autonomous operation that fundamentally distinguishes an AI agent from a simple prompt-response interface.

1.2 Agentic Workflow vs. Simple Automation

A critical distinction that practitioners must internalise is the difference between deterministic automation and agentic workflows. Conflating the two leads to architectural mismatches and suboptimal system design.

Deterministic Automation refers to any process that follows a fixed, pre-scripted sequence of steps. There is no decision-making involved — the system executes its prescribed script faithfully regardless of context. A canonical example is an automated email drip campaign:

1. A user subscribes to a mailing list.
2. The system dispatches a pre-written 'Welcome Email' immediately.
3. After a fixed interval of three days, a pre-written 'Follow-up Email' is dispatched.

This workflow is efficient and reliable for its singular, well-defined purpose, but it is entirely incapable of adapting to novel situations. Should a subscriber reply with a question, the deterministic system has no mechanism to formulate an intelligent response outside its script.

Agentic Workflows represent a fundamentally different paradigm. Instead of following prescribed steps, agents must decide which steps to take, dynamically, based on the specific nature of each task they encounter. The LLM provides the decision-making intelligence, but the architectural principle is this shift from a fixed script to a flexible, goal-oriented process capable of handling a broad range of inputs.

Key Insight

The defining characteristic of an agentic workflow is not the presence of an LLM — it is the system's ability to make contextual decisions about which actions to take next, rather than executing a pre-determined sequence. An LLM used solely for text completion within a fixed pipeline is not an agent; it is a sophisticated autocomplete engine.

2. Anatomy of an AI Agent: Core Components

To design effective agentic systems, one must understand the internal architecture of an agent. The following five components collectively define an agent's capabilities, identity, and operational boundaries. These are not merely conceptual abstractions — each component has direct implications for system design, prompt engineering, and infrastructure choices.

2.1 Persona

Definition: The Persona component defines the agent's identity, role, and operational character within the workflow. It is the mechanism by which a broad-purpose LLM is focused into a consistent, task-appropriate specialist.

In practice, the Persona is instantiated primarily through the system prompt — the foundational instruction set that precedes all user interactions. A well-designed persona communicates to the LLM:

- Its specific functional role (e.g., 'You are a Senior Financial Analyst specialising in equity markets').
- The appropriate communication tone and register (formal, technical, empathetic, concise, etc.).
- The precise boundaries of its domain of expertise and what falls outside its scope.
- The expected output format — structured JSON, markdown prose, bullet-point summaries — ensuring compatibility with downstream workflow components.

Consider the distinction between a 'Formal Legal Document Drafter' persona and a 'Friendly Customer Support Bot' persona applied to the same underlying LLM. Although both share identical base capabilities, the persona configuration transforms their outputs into entirely distinct artefacts suited for their respective contexts. This focused specialisation is critical for multi-agent architectures where different agents must produce outputs that are both independently useful and mutually compatible.

2.2 Knowledge

Definition: The Knowledge component encompasses all information sources an agent can access to perform its function. It extends well beyond the LLM's static training data.

An agent's effective knowledge base is composed of multiple layers:

- **Foundational LLM Training:** The extensive parametric knowledge encoded in the model's weights during pre-training. This constitutes the agent's 'world knowledge' baseline.

- **Fine-Tuning:** Domain-specific knowledge injected by training the model on curated datasets. This allows an agent to acquire specialised expertise beyond its general training.
- **External Information via Tools:** Real-time, dynamic knowledge retrieved on-the-fly from databases, REST APIs, web search engines, and internal knowledge bases. This is the primary mechanism for keeping agent knowledge current beyond the LLM's training cutoff date.
- **Memory:** Contextual information derived from current or prior interactions. Memory may be short-term (within a conversation window), long-term (persisted across sessions), or episodic (event-specific recall).

The architectural design of an agent's knowledge layer directly determines the accuracy, recency, and specificity of its outputs. An agent relying solely on parametric knowledge will hallucinate on time-sensitive queries; an agent with well-designed retrieval augmentation can remain accurate indefinitely.

2.3 Prompting Strategy

Definition: The Prompting Strategy is the systematic methodology by which the agent communicates with its underlying LLM. It is the operational blueprint that governs how tasks are framed, how context is structured, and how the LLM's reasoning capabilities are elicited.

A mature prompting strategy encompasses several distinct elements:

- **System Prompts:** Foundational instructions that establish the agent's persona and overarching objective for the duration of an interaction.
- **Context Incorporation:** The mechanism by which relevant information from memory stores, external tool outputs, or prior workflow steps is woven into the current prompt.
- **Prompt Engineering Techniques:** Advanced methodologies including Chain-of-Thought (CoT) prompting, which elicits explicit step-by-step reasoning; ReACT (Reasoning and Acting), which interleaves reasoning with tool invocation; and few-shot prompting, which provides illustrative examples to anchor the LLM's response patterns.
- **Output Specification:** Precise definition of the required response format, ensuring that the LLM's output is directly consumable by subsequent workflow components without additional parsing.

2.4 Execution / Tools

Definition: The Execution/Tools component provides the agent with its actuation capabilities — the ability to effect change in the world beyond generating text responses.

Tools transform an agent from a sophisticated text generator into an active participant in complex processes. Common tool categories include:

- Information retrieval tools: Web search engines, vector database queries, API calls for real-time data.
- Computational tools: Code interpreters, mathematical calculation engines, data transformation utilities.
- Persistence tools: Database write operations, file system interactions, CRM and ERP updates.
- Communication tools: Email dispatch, messaging platform integrations, notification systems.
- Orchestration tools: The ability to invoke other specialised agents, creating hierarchical multi-agent architectures.

The availability and design of tools is the single most significant factor determining the scope of tasks an agent can autonomously complete. Without execution capability, an agent can only advise; with well-designed tools, it can act, verify, and iterate.

2.5 Interaction

Definition: The Interaction component defines the communication protocols and interfaces by which an agent exchanges information with its operational environment — whether that environment consists of human users, other software systems, or peer agents.

The interaction layer governs three principal communication modalities:

- **Input Reception:** How the agent receives its task assignment, including the format of incoming data (natural language, structured JSON, API payloads) and the triggering mechanism (user message, scheduled event, upstream agent output).
- **Output Delivery:** How the agent delivers its results — as free-form text, structured data schemas, or by triggering downstream events or actions within the broader workflow.

- **Inter-Agent Communication:** The protocols governing agent-to-agent communication in multi-agent systems. This includes message passing formats, handoff conventions, and error signalling standards.

2.6 Component Summary Reference

Component	Primary Function	Key Design Consideration
Persona	Defines identity, role, tone, and output format	Specificity and consistency across all interactions
Knowledge	Provides access to information for task completion	Balancing parametric vs. retrieved knowledge
Prompting Strategy	Governs LLM communication and reasoning elicitation	Clarity, context management, and output specification
Execution / Tools	Enables real-world actions and data retrieval	Tool selection, error handling, and security boundaries
Interaction	Manages all external communication protocols	Format compatibility and inter-agent handoff design

3. Agent Types and the Autonomy Spectrum

Not all agents are architecturally equivalent. The manner in which an agent interacts with its LLM and manages context determines both its inherent capabilities and the complexity of workflows it can support. It is analytically useful to conceptualise agent types as existing on a spectrum of autonomy, ranging from simple, low-autonomy responders to sophisticated, fully autonomous problem-solvers.

3.1 Direct Prompting Agents

Autonomy Level: Low | **Context Handling:** Static (within-prompt) | **LLM Role:** Responder/Generator

The Direct Prompting Agent represents the most elementary architectural pattern. It operates as a transparent relay: a user's prompt is forwarded to the LLM with no augmentation, and the raw LLM response is returned directly to the caller. There is no memory, no tool access, and no contextual enrichment.

This agent type is appropriate for simple, single-turn workflows such as basic question-answering, text classification, or one-shot content generation where the prompt alone contains all necessary context.

3.2 Prompt Augmentation Agents

Autonomy Level: Low-Medium | **Context Handling:** Pre-augmented static context (e.g., RAG) | **LLM Role:** Guided Responder/Generator

Prompt Augmentation Agents enhance the incoming prompt before LLM invocation by injecting additional context. This augmentation may include a predefined persona (via system prompt), curated few-shot examples, or documents retrieved from a knowledge base. The injection of retrieved documents prior to LLM invocation is the defining feature of Retrieval-Augmented Generation (RAG) architectures.

This agent type is well-suited for specialised workflows requiring domain-specific expertise, such as legal document analysis, financial report generation, or technical support systems where proprietary knowledge bases must inform responses.

3.3 Dynamic Context Augmentation Agents

Autonomy Level: Medium | **Context Handling:** Dynamically updated via memory/tools | **LLM Role:** Adaptive Responder/Processor

Dynamic Context Augmentation Agents represent a significant increase in sophistication. These agents can invoke external tools and update the LLM's context in real-time during an interaction. The agent does not merely retrieve a static document — it actively queries external systems, processes the results, and uses this freshly acquired information to inform its subsequent reasoning and outputs.

This architecture is the foundational pattern for building dynamic, multi-step workflows. Consider an agent monitoring financial markets: upon receiving a query about a specific equity, it dynamically retrieves current stock prices via an API, processes the retrieved data, and synthesises a contextually relevant investment recommendation — all within a single, continuous workflow.

3.4 Autonomous Agents

Autonomy Level: High | **Context Handling:** Dynamic internal/external memory-driven | **LLM Role:** Planner/Executor/Reasoner

At the advanced end of the autonomy spectrum, Autonomous Agents leverage the LLM for complex, multi-step planning and decision-making with minimal human intervention. Upon receiving a high-level goal, an autonomous agent independently decomposes it into a sequence of sub-tasks, selects and invokes the appropriate tools for each, evaluates the results, self-corrects in the event of errors, and drives the entire workflow through to completion.

These agents serve as the orchestrators or primary executors of the most sophisticated agentic architectures, capable of handling unpredictable, open-ended problems that cannot be addressed through pre-scripted workflows.

3.5 Agent Types — Comparative Reference

Agent Type	LLM Role	Context Handling	Autonomy Level	Typical Use Case
Direct Prompting	Responder / Generator	Static (within prompt)	Low	Simple Q&A, basic text generation
Prompt Augmentation	Guided Responder / Generator	Pre-augmented static context (e.g., RAG)	Low–Medium	RAG-based Q&A, few-shot tasks
Dynamic Context Augmentation	Adaptive Responder / Processor	Dynamically updated via memory/tools	Medium	Multi-turn conversations, real-time tool use
Autonomous Agent	Planner / Executor / Reasoner	Dynamic internal/external memory-driven	High	Complex task completion, independent problem solving

4. Agentic Workflow Modeling

Having established the anatomy and taxonomy of agents, the next challenge is architectural: how does one systematically design and represent complex multi-agent systems? Agentic Workflow Modeling is the discipline concerned with this design process.

4.1 From Deterministic Automation to Agentic Systems

A practical approach to developing agentic systems is to begin with an existing deterministic process and progressively generalise it toward autonomy. This evolutionary strategy offers several advantages: it grounds the design in a well-understood process, provides a clear baseline for performance comparison, and allows incremental introduction of complexity.

The initial phase maps each step of the deterministic workflow to an agent. At this stage, the system remains largely deterministic — agents execute their assigned steps in sequence, without significant decision-making latitude. This is an acceptable and valuable starting point.

The transformative leap occurs during generalisation. Rather than hardcoding a fixed processing sequence, the system is enhanced with:

- **A Planning Agent:** A supervisory agent that analyses incoming requests and dynamically determines the appropriate processing path and agent allocation.
- **Expanded Tool Access:** Provision of diverse information retrieval and action execution tools that enable agents to respond to a broader class of inputs.
- **Decision-Making Frameworks:** Explicit logic governing how agents evaluate options and select actions, transforming reactive systems into proactive problem-solvers.

Example: The Evolving Code Assistant

Stage 1 (Deterministic): A Python syntax Q&A bot that answers only one type of question.

Stage 2 (Agentic): A generalised assistant with planning capability (identifies question type: Python, Java,

debugging, feature design), tool access (code repository search, web search, code validator), and

decision-making power (determines whether to answer from knowledge, invoke a tool, or seek clarification).

This transformation converts a fixed-process follower into an intelligent, adaptive system.

4.2 Agentic Workflows vs. Conversational Chatbots

A fundamental architectural distinction must be drawn between agentic workflows and conversational chatbots. Agentic workflows are designed to accomplish specific, bounded objectives — they have a defined commencement point, a structured processing pipeline, and a determinate conclusion upon task completion. Conversational chatbots, by contrast, are designed for open-ended, ongoing dialogue with no predefined terminus.

This distinction carries significant design implications. Agentic workflows must be explicitly designed with completion criteria, error propagation management, and output validation — concerns that are secondary in conversational architectures.

Critically, agentic workflows retain an important degree of flexibility absent in purely deterministic systems:

- **Reflective Iteration:** Agents can review and critique their own outputs or those of peer agents before proceeding, enabling quality improvement loops.
- **Path Diversity:** Unlike deterministic pipelines, agentic workflows can discover and traverse alternative execution paths to achieve a goal more efficiently.

4.3 Process-Centric vs. Agent-Centric Modeling

The modeling paradigm appropriate for a workflow depends fundamentally on whether it is deterministic or agentic in nature.

Modeling Dimension	Deterministic (Process-Centric)	Agentic (Agent-Centric)
Primary Focus	Task sequence and inter-task dependencies	Agent capabilities and goal structures
Decision Points	Fixed conditional rules (if/else logic)	Dynamic LLM-driven decision frameworks
Adaptability	None — script is immutable	High — paths adapt to input characteristics
Diagram Style	Traditional flowchart	Agent interaction diagram with capability nodes
Context Management	Not applicable	Explicit memory, feedback loops, and context passing

4.4 Visual Modeling with Mermaid.js

Communicating complex multi-agent architectures demands effective visual representation. Mermaid.js is a widely adopted JavaScript-based diagramming library that generates visual diagrams from human-readable text syntax, analogous in approach to Markdown for text formatting.

Mermaid.js is particularly well-suited for agentic workflow documentation because:

- Diagrams are defined as text, enabling version control alongside source code.
- The syntax is concise and learnable, lowering the barrier to diagram creation.
- It supports a comprehensive range of diagram types including flowcharts, sequence diagrams, state diagrams, class diagrams, and Gantt charts.
- Diagrams update in real-time in supported editors and online environments.

When constructing visual models of agentic workflows, the following notational principles should be observed:

- **Standard Symbols:** Rectangles represent processing tasks or agent operations; diamonds represent decision points; parallelograms represent input/output; cylinders represent data stores.
- **Descriptive Labeling:** Node labels should be action-oriented and specific. 'Analyse Customer Feedback Sentiment' is preferable to 'Analyse Data'.
- **Explicit Data Flow:** Arrows should indicate not merely control flow but the nature of the data being transferred between nodes.
- **Appropriate Granularity:** High-level diagrams for system overview communication; detailed sub-flow diagrams for implementation guidance.

4.5 Common Agent Archetypes as Modeling Building Blocks

When modeling agentic systems, it is useful to have a standard vocabulary of agent archetypes that can serve as reusable building blocks. The following represent the most frequently encountered patterns:

Agent Archetype	Description
Direct Prompt Agent	Forwards user query to LLM without augmentation; suitable for single-turn tasks.
Augmented Prompt Agent	Injects system prompt (persona/instructions) before LLM invocation to shape output style.
Knowledge Augmented Agent	Uses a curated, specialised knowledge base; ignores LLM's general parametric knowledge.
RAG Agent	Dynamically retrieves relevant documents from a large corpus via semantic search before responding.
Evaluation Agent	Acts as a quality controller, assessing peer agent outputs against defined criteria.
Routing Agent	Classifies incoming tasks and dispatches them to the most appropriate specialised agent.

Action Planning Agent	Decomposes complex goals into ordered sequences of executable sub-tasks.
-----------------------	--

5. Core Agentic Workflow Patterns

Agentic systems are rarely monolithic. In practice, they are composed of recurring architectural patterns that address specific classes of challenges. Mastery of these patterns is essential for designing robust, production-grade agentic applications. This section formally describes the four foundational patterns.

5.1 Pattern I: Prompt Chaining

Core Principle: Decompose a complex task into a sequence of simpler, ordered sub-tasks, where the output of each LLM call serves as the input for the next.

Prompt chaining operationalises the principle of task decomposition for LLM-based systems. The motivation is straightforward: LLMs produce more accurate, reliable outputs when addressing focused, well-scoped tasks rather than attempting to solve complex, multi-faceted problems in a single invocation. By structuring a complex task as a sequential assembly line — where each station performs one specialised operation — the overall system achieves greater accuracy and interpretability.

⚠️ Primary Risk: Error Propagation

The sequential dependency structure of prompt chaining creates a systemic vulnerability: a flawed output from an early step propagates downstream and may compound through subsequent steps. A small error in the initial data extraction phase of a financial report chain can cascade into fundamentally incorrect calculations, analyses, and recommendations throughout the entire workflow.

Intermediate Validation Strategies

The primary defence against error propagation is systematic intermediate validation — the insertion of quality control gates between chain steps. Four principal validation approaches are available:

Validation Strategy	Description & Appropriate Use Cases
---------------------	-------------------------------------

Programmatic Checks	Custom code validates specific, deterministic conditions: JSON schema compliance, numerical range validation, string length constraints, format matching via regex.
LLM-Based Validation	A secondary LLM call evaluates the output of the primary call for subjective or semantic qualities: factual accuracy, tonal appropriateness, logical coherence, and relevance.
Rule-Based Validation	Predefined, explicitly declared rules check for required patterns: presence of mandatory fields, adherence to structural templates, compliance with business logic.
Confidence Scoring	The generating LLM's confidence scores (where available) are checked programmatically against a minimum threshold; low-confidence outputs trigger remediation.

Error Remediation Strategies

When a validation gate identifies a failed intermediate output, the following remediation strategies are available, in order of escalating complexity:

4. **Retry:** Re-invoke the failing prompt unchanged. Effective for transient LLM variability.
5. **Re-Prompt with Feedback:** Incorporate the specific failure reason into a revised prompt, directing the LLM to correct identified deficiencies.
6. **Critique and Refinement Loop:** Prompt the LLM to self-critique its output against specified criteria, then prompt it to revise based on that critique.
7. **Fallback:** Terminate the chain, log the error with full context, and route to a predefined safe fallback response or escalation path.

Context Management in Chained Workflows

Effective context management is a non-trivial engineering challenge in prompt chaining. Two opposing risks must be managed simultaneously: providing insufficient context (causing the LLM to lose relevant earlier information) and providing excessive context (exceeding the context window limit or degrading response quality through irrelevant noise).

The recommended approach is **Selective Context Passing** — transmitting only the most relevant sub-sections of prior outputs to subsequent steps. Where critical information from early steps must be preserved across many downstream steps, **Contextual Reiteration** — restating key facts explicitly within each prompt — is an effective technique.

5.2 Pattern II: Routing

Core Principle: Intelligently classify incoming tasks and dispatch them to the specialised agent or processing path best equipped to handle each task's specific characteristics.

The Routing pattern is the architectural solution to the challenge of task heterogeneity. In real-world deployments, systems must handle a diverse range of requests that differ in topic, complexity, intent, and required expertise. Rather than processing all requests through a single, generalised pipeline — which inevitably compromises either capability or efficiency — the routing pattern implements a smart dispatch mechanism analogous to a sophisticated telecommunications exchange or a multi-speciality hospital's triage system.

The routing pattern delivers four key architectural benefits:

- **Task Specialisation:** Directing inputs to purpose-optimised agents or prompts produces superior accuracy compared to general-purpose processing.
- **Resource Optimisation:** Routing simpler tasks to faster, less expensive models while reserving powerful (and costly) models for genuinely complex tasks optimises computational expenditure.
- **Flexibility and Adaptability:** The system accommodates diverse request types dynamically without architectural modification.
- **Scalability:** New specialised agents can be added to the routing topology with minimal disruption to the existing system.

Stage 1: Classification

Classification is the analytical stage in which the system examines an incoming task to determine its type, category, intent, or complexity. Accurate classification is the single most critical factor for routing system performance — a misclassified input dispatched to the wrong specialist agent will produce irrelevant or incorrect outputs regardless of that agent's capability.

Classification may be implemented through two primary approaches:

- **Rule-Based Classification:** Deterministic logic — keyword matching, pattern recognition, conditional statements — applied to the input. Fast and computationally inexpensive; suitable for inputs with clear, distinguishable features.
- **LLM-Based Classification:** A dedicated LLM call analyses the input and assigns a category label based on semantic understanding. Slower and more expensive than rule-

based approaches; necessary for ambiguous or nuanced inputs where surface-level pattern matching is insufficient.

Stage 2: Task Dispatch

Following classification, Task Dispatch implements the branching logic that directs the classified input — along with its category label — to the designated specialist agent or processing module. The dispatch mechanism functions as a conditional routing table: if the classification is 'Technical Support Query', dispatch to the Technical Support Agent; if 'Billing Inquiry', dispatch to the Billing Agent; and so forth. Each downstream agent is designed with the capabilities and knowledge precisely matched to its designated input category.

5.3 Pattern III: Parallelisation

Core Principle: Distribute workloads across multiple agents operating concurrently, then aggregate their independent outputs into a unified final result.

The Parallelisation pattern — also known as the scatter-gather pattern in distributed computing terminology — addresses two distinct but related challenges: latency reduction for large tasks and quality enhancement through diverse perspective generation.

Prerequisite: Task Independence

Effective parallelisation requires that constituent sub-tasks be largely independent of one another. Sub-task A must not require the output of Sub-task B as a prerequisite for its own execution. Where dependencies exist between sub-tasks, prompt chaining (sequential processing) is the appropriate pattern for those dependent components.

Task Decomposition Strategies

Three principal strategies govern how a complex task is decomposed into parallelisable sub-tasks:

Strategy	Description
Sectioning / Sharding	Partition a large, divisible input (long document, large dataset) into equal-sized chunks, each processed by a separate agent concurrently.

Aspect-Based Decomposition	Different agents concurrently analyse distinct facets of a single subject: one agent evaluates technical specifications, another examines user sentiment, a third assesses competitive positioning.
Identical Tasks for Diversity / Voting	The same core task is executed multiple times in parallel, with different prompts, temperatures, or models, to generate diverse outputs or enable majority-vote consensus for higher reliability.

Output Aggregation Strategies

Once parallel sub-tasks complete, their outputs must be consolidated. Four aggregation strategies are commonly employed:

Aggregation Strategy	Description
Concatenation	Direct sequential joining of parallel outputs; appropriate when outputs represent distinct, non-overlapping portions of the final result.
Comparison and Selection	Parallel outputs are evaluated against predefined quality criteria; the highest-scoring output is selected as the final result.
Voting / Majority Rule	Multiple agents independently produce the same classification or decision; the most frequently occurring answer is adopted. Improves reliability on ambiguous tasks.
Synthesiser LLM	A dedicated aggregator LLM receives all parallel outputs and synthesises them into a single, coherent, and nuanced final response, resolving contradictions and filling gaps.

5.4 Pattern IV: Evaluator-Optimizer

Core Principle: Establish a structured iterative cycle of generation and critique, in which an Optimizer agent produces outputs and an Evaluator agent assesses and provides actionable feedback, driving progressive quality improvement.

The Evaluator-Optimizer pattern is the architectural response to the inherent quality limitations of single-pass LLM generation. It is particularly indispensable for tasks requiring high precision, rigorous constraint adherence, and top-tier quality — such as production code generation, legal document drafting, comprehensive research reports, or precision marketing materials.

The pattern defines two complementary agent roles:

- The Optimizer (Generator) Agent:** Receives the original task specification and generates an initial output. Upon receiving evaluative feedback, it revises its output to address identified deficiencies, guided by the feedback's specific directives.

- **The Evaluator (Critique/Reflector) Agent:** Operates as an expert peer reviewer. It assesses the Optimizer's output against a predetermined set of evaluation criteria and produces structured, actionable feedback specifying precise improvements.

Critical Design Elements

Three elements determine the effectiveness of an Evaluator-Optimizer implementation:

Design Element	Requirements for Effectiveness
Clear Evaluation Criteria	Criteria must be specific, measurable, and unambiguous. Vague criteria produce vague feedback, which produces vague revisions. Examples: 'all references must be cited in APA format', 'code must pass all unit tests', 'summary must be under 150 words'.
Actionable Feedback	Evaluator feedback must specify precisely what is deficient and provide concrete direction for improvement. 'Improve clarity' is insufficient; 'The third paragraph's causal claim lacks supporting evidence — add at least one citation' is actionable.
Stopping Conditions	The iterative loop must have clearly defined termination criteria to prevent infinite loops: (1) all evaluation criteria satisfied, (2) maximum iteration count reached, (3) improvement delta between cycles falls below defined threshold.

5.5 Pattern V: Orchestrator-Workers

Core Principle: A central Orchestrator agent dynamically analyses complex tasks, decomposes them into sub-tasks at runtime, delegates those sub-tasks to specialised Worker agents, and synthesises their outputs into a coherent final result.

The Orchestrator-Workers pattern represents the most architecturally sophisticated of the core patterns, enabling truly dynamic, adaptive problem-solving. It is the pattern of choice when task complexity is high, solution paths are unpredictable in advance, and specialist expertise must be coordinated intelligently.

The defining distinction between the Orchestrator-Workers pattern and simple parallelisation is the presence of dynamic intelligence: the Orchestrator does not follow a fixed decomposition blueprint — it analyses each incoming task at runtime and makes autonomous decisions about the necessary sub-tasks and optimal agent assignments.

The Orchestrator's Dual Function

- **Dynamic Decomposition and Delegation:** Upon receiving a high-level goal, the Orchestrator analyses its structure, identifies the necessary specialist capabilities, and dispatches customised sub-task assignments to appropriate Worker agents. This is a runtime decision process, not a pre-scripted assignment.
- **Synthesis:** Upon receiving completed sub-task outputs from all Worker agents, the Orchestrator integrates these disparate pieces into a single, coherent, and holistic final output — connecting insights from different domains to produce emergent understanding beyond what any single Worker could generate.

Orchestrator-Workers vs. Parallelisation: A Critical Distinction

Dimension	Orchestrator-Workers	Simple Parallelisation
Intelligence	Dynamic — runtime decomposition	Static — fixed pre-defined decomposition
Adaptability	High — adapts to novel problem structures	Low — suited to known, repeatable workloads
Orchestration	Active management, delegation, synthesis	Distribute and collect
Best Suited For	Complex, unpredictable, open-ended problems	Repetitive, divisible, predictable workloads
Analogy	Smart Project Manager with specialist team	Assembly line with parallel stations

6. Implementing Agentic Workflows in Python

Having established the theoretical architecture of agentic systems, this section addresses the practical engineering question: how does one translate these abstract patterns into concrete, maintainable Python code? The fundamental principles of modularity, separation of concerns, and object-oriented design — standard pillars of software engineering — are directly applicable and highly valuable in this domain.

6.1 Architectural Design Philosophy

Two primary implementation paths are available to practitioners:

- **Framework-Based Development:** Leverage established agentic frameworks (LangChain, LlamaIndex, CrewAI, AutoGen) that provide pre-built abstractions for

common patterns. Advantageous for rapid prototyping and access to an extensive ecosystem of integrations.

- **From-Scratch Implementation:** Construct agent logic directly using Python and LLM API calls. More labour-intensive, but provides complete transparency, maximum customisation, and deep architectural understanding — invaluable for learning and for highly specialised production requirements.

The from-scratch approach, while demanding, is particularly recommended during the learning phase, as it ensures practitioners understand the precise mechanics of agent operation rather than relying on framework abstractions that may obscure underlying behaviour.

6.2 Python Code Structure for Agentic Systems

A well-architected Python codebase for an agentic system follows a three-layer modular structure that enforces separation of concerns and promotes extensibility:

8. Agent Library / Module (**agent_definitions.py**): A dedicated Python module containing class definitions for each agent type. Each class encapsulates the agent's attributes (persona, model configuration, available tools) and methods (prompt construction, LLM invocation, output parsing). This layer defines what agents are.
9. Agent Instantiation Logic (**workflow_setup.py**): A script that imports agent classes and creates specific, configured instances for a given workflow. This is where agents are customised for their designated roles — specifying the appropriate persona, model, and tool subset for each instance. This layer defines which agents exist in a given workflow.
10. Workflow Orchestration Logic (**main.py**): The top-level script that imports agent instances and implements the workflow's execution logic — the sequence of agent invocations, data passing between agents, conditional routing, error handling, and output aggregation. This layer defines how agents collaborate to achieve the workflow goal.

E Architectural Principle: Separation of Concerns

Separating agent definitions (what), instantiation (which), and orchestration (how) into distinct modules

provides three critical engineering benefits: (1) Reusability — the same agent class can be instantiated with

different configurations for different workflows; (2) Testability — each layer can be unit-tested independently;

(3) Maintainability — modifications to agent behaviour require changes in only one location.

SECTION II: PRACTICAL IMPLEMENTATION & CODE

This section provides comprehensive, annotated implementations of the theoretical concepts introduced in Section I. Each code block is presented as it would appear in an IPython/Jupyter Notebook cell, followed by a detailed explanation of its purpose, design rationale, and expected behaviour. All implementations use Python 3.10+ and the OpenAI-compatible API interface.

Notebook Cell 1: Environment Setup and Library Imports

Code

```
# — Cell 1: Environment Setup — #
Install required libraries (run once in your environment) # !pip install openai
python-dotenv import os import json import time from typing import Optional, List,
Dict, Any from dotenv import load_dotenv from openai import OpenAI # Load
environment variables from .env file load_dotenv() # Initialise the OpenAI client
with the API key from environment client =
OpenAI(api_key=os.getenv("OPENAI_API_KEY")) # Global model configuration
DEFAULT_MODEL = "gpt-4o-mini" # Cost-efficient default; swap to gpt-4o for
complex tasks DEFAULT_TEMP = 0.7 # Controls response randomness
(0=deterministic, 1=creative) print("Environment initialised successfully.")
print(f"Active model: {DEFAULT_MODEL}")
```

Explanation

This initialisation cell establishes all prerequisites for subsequent notebook cells. It is structured to follow the standard Python project best practice of separating configuration from logic.

- **Library Imports:** The `openai` library provides the Python SDK for interacting with GPT models. `python-dotenv` enables loading sensitive credentials from a `.env` file rather than hardcoding them in source code — a critical security practice in any project that may be version-controlled.
- **OpenAI Client Initialisation:** The `OpenAI()` client object is the primary interface for all subsequent LLM API calls. Storing the API key in an environment variable (retrieved via `os.getenv()`) ensures it is never exposed in the notebook source.
- **Model Configuration Constants:** `DEFAULT_MODEL` and `DEFAULT_TEMP` are defined as module-level constants, enabling consistent configuration across all agent implementations and facilitating easy model substitution during experimentation.
- **Temperature Parameter:** A temperature of 0.7 represents a commonly effective starting point, balancing coherence with appropriate diversity. For validation tasks, a lower

temperature (0.1–0.3) is recommended to promote deterministic, consistent assessments; for creative generation, higher values (0.8–1.0) may be preferred.

Notebook Cell 2: The Base Agent Class Definition

Code

```
# — Cell 2: Base Agent Class —
class BaseAgent: """ Abstract base class for all agent implementations.
Encapsulates core LLM invocation logic and provides a consistent interface for
all derived agent types. """
    def __init__(self, name: str,
                  persona: str, model: str = DEFAULT_MODEL, temperature:
float = DEFAULT_TEMP, max_tokens: int = 1500, ):
        self.name = name
        self.persona = persona
        self.model = model
        self.temperature = temperature
        self.max_tokens = max_tokens
    def _invoke_llm(self, user_message: str, additional_context:
Optional[str] = None, ) -> str: """ Core method: constructs
message payload and invokes the LLM. Returns the raw text content of the
model's response. """
        system_content = self.persona
        if additional_context:
            system_content += f"\n\nAdditional
Context:\n{additional_context}"
        messages = [ {"role": "system", "content": system_content},
                      {"role": "user", "content": user_message},
                    ]
        response = client.chat.completions.create(
            model=self.model, messages=messages,
            temperature=self.temperature, max_tokens=self.max_tokens,
        )
        return response.choices[0].message.content
    def run(self, task: str, context:
Optional[str] = None) -> str: """ Public interface for executing the agent's
primary task. """
        print(f"[{self.name}] Processing task...")
        result = self._invoke_llm(task, context)
        print(f"[{self.name}] Task complete.")
        return result
```

Explanation

The BaseAgent class is the foundational building block of the entire agentic system. Its design reflects core object-oriented programming principles, particularly the template method pattern, which defines the skeleton of an algorithm in a base class while deferring specific implementation details to subclasses.

- **__init__ Constructor:** The constructor accepts five parameters that collectively define the agent's identity and operational characteristics. The name parameter serves both logging/debugging purposes and semantic identification within multi-agent systems. The persona parameter receives the system prompt string — the single most important configuration for determining agent behaviour.
- **_invoke_llm Method (Private):** The leading underscore follows Python convention for private methods — this method is for internal use only and not part of the public API. It constructs the messages list in the format required by the OpenAI Chat Completions API: a system message containing the persona (and optionally supplementary context) and a user message containing the task. The system message is processed before the user message, giving the persona instructions precedence over the task request.

- **additional_context Parameter:** This optional parameter is the mechanism through which dynamic information — retrieved documents, prior step outputs, memory contents — can be injected into the agent's context window without modifying the persona. This clean separation between stable identity (persona) and dynamic context is an important architectural pattern.
- **run Method (Public):** The public interface that external code calls to execute an agent's task. The print statements provide basic console logging for operational visibility — in production systems, these would be replaced by a structured logging framework (e.g., Python's logging module).

Notebook Cell 3: Implementing the Prompt Chaining Pattern

Code

```
# — Cell 3: Prompt Chaining Implementation —————
import re
class PromptChainingWorkflow:
    """ Implements a sequential prompt
    chaining workflow with intermediate programmatic and LLM-based validation.
    Demonstrates a three-step content creation pipeline:
    1. Research Outline Generation
    2. Detailed Draft Expansion
    3. Professional Polish and Formatting
    """
    def __init__(self):
        # Step 1: Outline agent — structured thinking
        self.outliner = BaseAgent(
            name="Outliner",
            persona=(
                "You are an expert content strategist. When given a "
                "topic, "
                "produce a structured outline with exactly 4 main "
                "sections, "
                "each with 2-3 bullet sub-points. Respond ONLY with the "
                "outline."
            ),
            temperature=0.4,
        )
        # Step 2: Drafting agent — prose expansion
        self.drafter = BaseAgent(
            name="Drafter",
            persona=(
                "You are a technical writer. "
                "Expand the provided outline into "
                "well-structured prose "
                "paragraphs. Write at least 150 words per section. "
                "Maintain an "
                "academic, formal tone throughout."
            ),
            temperature=0.7,
        )
        # Step 3: Polish agent — final quality pass
        self.polisher = BaseAgent(
            name="Polisher",
            persona=(
                "You "
                "are a senior editor. Review the provided draft and: "
                "(1) Correct "
                "any grammatical issues, "
                "(2) Improve sentence variety, "
                "(3) Ensure consistent academic tone. "
                "Return the polished final "
                "text."
            ),
            temperature=0.3,
        )
        # Validation agent — LLM-based quality gate
        self.validator = BaseAgent(
            name="Validator",
            persona=(
                "You are a quality "
                "assurance reviewer. Assess whether the "
                "provided content meets "
                "the specified criteria. "
                "Respond ONLY with a JSON object: "
                '{"passed": true/false, "reason": "explanation", '
                '"score": 1-10}'
            ),
            temperature=0.1, # Near-deterministic for consistent evaluation
        )

        def _validate_outline(self, outline: str) -> Dict[str, Any]:
            """Programmatic validation: checks outline has minimum structure."""
            sections = [l for l in outline.split("\n") if l.strip() and
                        l.strip().startswith("-") and not l.strip().startswith("•")]
            if len(sections) < 3:
                return {"passed": False, "reason": "Outline has "
                    "fewer than 3 main sections."}
            if len(outline.split("\n")) < 10:
                return {"passed": False, "reason": "Outline lacks sufficient sub-points."}
            return {"passed": True, "reason": "Outline structure is satisfactory."}

        def _validate_draft_quality(self, draft: str, outline: str) -> Dict[str, Any]:
            """LLM-based validation: assesses draft quality against outline."""
            prompt = (
                f"Outline provided:\n{outline}\n\n"
                f"Draft
```



```

produced:\n{draft}\n\n"
outline sections? "
and academic?"
)
# Strip markdown code fences if LLM wraps JSON in them
return json.loads(clean)
except json.JSONDecodeError:
    return {"passed": True, "reason": "Validation parse error; proceeding.", "score": 7}
def execute(self, topic: str, max_retries: int = 2) -> Optional[str]:
    """
    Executes the full three-step prompt chaining workflow.
    Returns the final polished content, or None if the chain fails.
    """
    print(f"\n{'='*60}")
    print(f" PROMPT CHAINING WORKFLOW: {topic}")
    print(f"{'='*60}")
    # — STEP 1: Generate Outline
    print("\n[Step 1/3] Generating structured outline...")
    outline = self.outliner.run(f"Create a detailed outline for: {topic}")
    # Programmatic validation gate
    outline_check = self._validate_outline(outline)
    if not outline_check["passed"]:
        print(f" ⚠ Outline validation failed: {outline_check['reason']}")
        for attempt in range(max_retries):
            print(f" Retry {attempt + 1}/{max_retries}...")
            outline = self.outliner.run(f"Create a detailed outline for: {topic}. "
                                   f"IMPORTANT: Ensure exactly 4 main sections with sub-points."
                                   )
            outline_check = self._validate_outline(outline)
            if outline_check["passed"]:
                break
        else:
            print(" X Outline generation failed after all retries. Aborting.")
    return None
    print(f" ✓ Outline validated successfully.")
    # — STEP 2: Expand to Draft
    print("\n[Step 2/3] Expanding outline into full draft...")
    draft = self.drafter.run("Expand the following outline into comprehensive prose:",
                             context=outline
                             )
    # LLM-based validation gate
    draft_check = self._validate_draft_quality(draft, outline)
    print(f" Draft quality score: {draft_check.get('score', 'N/A')}/10")
    if not draft_check.get("passed", True):
        print(f" ⚠ Draft validation failed: {draft_check.get('reason')}")
        print(" Re-prompting drafter with feedback...")
        draft = self.drafter.run(f"Expand the following outline into comprehensive prose. "
                                  f"FEEDBACK TO ADDRESS: {draft_check.get('reason')}",
                                  context=outline
                                  )
        print(f" ✓ Draft accepted.")
    # — STEP 3: Polish
    print("\n[Step 3/3] Applying editorial polish...")
    final = self.polisher.run("Polish the following draft to publication standard:",
                              context=draft
                              )
    print(" ✓ Polish complete.\n")
    return final
    # — Execution
    workflow = PromptChainingWorkflow()
    result = workflow.execute("The Architecture of Modern AI Agent Systems")
    if result:
        print("\n" + "="*60)
        print(" FINAL OUTPUT")
        print("="*60)
        print(result[:800] + "..." if len(result) > 800 else result)

```

Explanation

This cell provides a complete, annotated implementation of the Prompt Chaining pattern applied to a content creation pipeline. The workflow decomposes the complex task of producing a polished article into three focused, sequential sub-tasks, each handled by a specialised agent instance.

- **Agent Specialisation via Temperature:** Observe that each agent is configured with a different temperature value reflecting its task's requirements. The Outliner uses 0.4 (low-moderate) to encourage structured, consistent outputs. The Drafter uses 0.7 (moderate) to balance coherence with creative prose quality. The Polisher uses 0.3 (low) to promote

precise, consistent editorial decisions. The Validator uses 0.1 (near-deterministic) to ensure reliable, reproducible quality assessments.

- **Dual Validation Strategy:** The implementation demonstrates both validation approaches described in Section 5.1. The `_validate_outline` method performs fast, programmatic validation by counting structural elements — appropriate for objective structural requirements that can be checked with code. The `_validate_draft_quality` method performs LLM-based validation to assess subjective qualities (completeness, tone, length adequacy) that resist algorithmic checking.
- **Retry with Feedback:** When outline validation fails, the retry logic incorporates explicit corrective instructions into the revised prompt ('Ensure exactly 4 main sections'). This targeted re-prompting is considerably more effective than blindly re-running the identical prompt.
- **Context Passing via the context Parameter:** In Step 2, the outline is passed as `additional_context` to the Drafter. This injects the outline into the system prompt extension (see `BaseAgent._invoke_llm`), providing the Drafter with the structural blueprint it needs without embedding it in the task instruction — maintaining clean separation between the task (what to do) and the context (what information to use).
- **JSON Response Parsing with Defensive Handling:** The LLM validation step parses a JSON response. The `re.sub()` call strips markdown code fences (````json ... ````) that LLMs sometimes add around JSON blocks. The `try/except` block catches JSON parse errors gracefully, defaulting to a pass decision to avoid blocking the workflow on a validator failure — a pragmatic production consideration.

Notebook Cell 4: Implementing the Routing Pattern

Code

```
# — Cell 4: Routing Pattern Implementation —————
class RoutingWorkflow:
    """ Implements the two-stage routing pattern:
    Classification + Task Dispatch. Demonstrates an intelligent customer support
    routing system with three specialist agents and an LLM-based classifier.
    """
    # Define routing categories and their specialist agents
    CATEGORIES = {
        "billing": "Billing Specialist", "technical": "Technical
        Support Engineer", "general": "General Support Agent",
        "escalation": "Senior Account Manager", }
    def __init__(self):
# — Stage 1: Classifier —————
        self.classifier = BaseAgent(
            name="Classifier", persona=(
                "You are a customer support triage specialist. "
                "Classify the incoming customer query into EXACTLY ONE category. "
                "Available categories: billing, technical, general, escalation. "
                "Respond ONLY with a JSON object: "
                '{"category": "<category>", "reasoning": "<brief explanation>"}'
            ), temperature=0.1, )
# — Stage 2: Specialist Worker
        self.workers = { "billing":
```

```
BaseAgent(
    name="Billing Specialist",
    persona=(
        "You are an expert billing and payments specialist. "
        "billing queries with precise, factual information. "
        "verify account information would be required before making changes. "
        "Maintain a professional, reassuring tone."
    ),
    temperature=0.5,
    "technical": BaseAgent(
        name="Technical Support Engineer",
        persona=(
            "You are a senior technical support engineer. "
            "systematic, step-by-step troubleshooting guidance. "
            "diagnostic clarifying questions when necessary. "
            "technical language appropriate for an intermediate user."
        ),
        temperature=0.4,
        "general": BaseAgent(
            name="General Support Agent",
            persona=(
                "You are a friendly, knowledgeable general customer support agent. "
                "Handle general enquiries helpfully and warmly. "
                "exceeds your scope, acknowledge this and offer to escalate."
            ),
            temperature=0.7,
            "escalation": BaseAgent(
                name="Senior Account Manager",
                persona=(
                    "You are a senior account manager handling complex, sensitive, "
                    "high-priority customer concerns. Demonstrate empathy, ownership, "
                    "and commitment to resolution. Provide clear next steps and timelines."
                ),
                temperature=0.5,
            ),
            def
            _classify(self, query: str) -> Dict[str, Any]:
                """Stage 1: LLM-based
                Classification of incoming query."""
                raw = self.classifier.run(query)
                try:
                    clean = raw
                    result = json.loads(clean)
                except:
                    # Validate that classified category is a known route
                    if result.get("category") not in self.CATEGORIES:
                        result["category"] = "general"
                    # Safe fallback
                    return result
                except json.JSONDecodeError:
                    return {"category": "general", "confidence": 0.5,
                            "reasoning": "Classification parse error; defaulting to general."}
                def
                _dispatch(self, category: str, query: str) -> str:
                    """Stage 2: Route
                    classified query to appropriate specialist."""
                    worker = self.workers.get(category, self.workers["general"])
                    return worker.run(
                        query,
                        context=f"This query has been classified as a '{category}'
                        enquiry."
                    )
                def handle(self, customer_query: str) -> Dict[str, Any]:
                    """
                    Main entry point. Classifies and dispatches a customer query.
                    Returns a structured response dict with routing metadata.
                    """
                    print(f"\n{'-'*55}")
                    print(f" Incoming Query: {customer_query[:60]}...")
                    print(f"\n{'-'*55}")
                    # Stage 1: Classify
                    classification = self._classify(customer_query)
                    category = classification["category"]
                    confidence = classification.get("confidence", 0.0)
                    reasoning = classification.get("reasoning", "")
                    print(f" Classification : {category.upper()} " f"(confidence: {confidence:.0%})")
                    print(f" Reasoning : {reasoning}")
                    print(f" Routing to : {self.CATEGORIES[category]}")
                    # Stage 2: Dispatch
                    response = self._dispatch(category, customer_query)
                    return {
                        "query": customer_query,
                        "category": category,
                        "confidence": confidence,
                        "agent": self.CATEGORIES[category],
                        "response": response,
                    }
                    # — Test with sample queries
                    router = RoutingWorkflow()
                    test_queries = [
                        "I've been charged twice for my subscription this month and need a refund urgently.",
                        "My application crashes every time I try to upload a file larger than 10MB.",
                        "What are your office hours and how can I contact you by phone?",
                    ]
                    for query in test_queries:
                        result = router.handle(query)
                    print(f"\n Response Preview: {result['response'][:200]}...\n")
```

Explanation

This cell implements a production-relevant customer support routing system that cleanly demonstrates the Classification → Task Dispatch two-stage architecture described in Section 5.2.

- **CATEGORIES Dictionary as a Routing Table:** The class-level CATEGORIES dictionary serves as the system's routing table — a declarative mapping from category labels to human-readable agent names. This externalises the routing configuration, making it straightforward to add, remove, or rename categories without modifying the routing logic itself.
- **Classifier Temperature = 0.1:** The classification agent uses a near-deterministic temperature configuration. Consistency and reproducibility are paramount for routing: the same query submitted twice should receive the same category label. Higher temperatures introduce stochastic variability that could produce inconsistent routing decisions.
- **The workers Dictionary:** Worker agents are stored in a dictionary keyed by category name, enabling O(1) dispatch lookup. Each agent's persona is precisely tailored to its designated domain — the Technical Support Engineer uses systematic troubleshooting language; the Billing Specialist emphasises verification and accuracy; the Senior Account Manager demonstrates empathy and ownership.
- **Safe Fallback Pattern:** The `_classify` method validates that the LLM's returned category exists in CATEGORIES. If an unknown category is returned (a possible LLM confabulation), it defaults to 'general' — the safest, most capable of handling diverse inputs. This defensive programming pattern is essential in production deployments.
- **Context Injection in `_dispatch`:** When dispatching to a worker, the category classification is explicitly passed as context ('This query has been classified as a billing enquiry'). This meta-information allows the worker agent to frame its response with appropriate domain focus from the outset.
- **Structured Return Dict:** The `handle` method returns a structured dictionary containing not just the response but the full routing metadata (category, confidence, agent name). This is consistent with production API design principles: callers receive rich contextual information about how a request was processed, enabling audit logging, monitoring, and downstream logic.

Notebook Cell 5: Implementing the Evaluator-Optimizer Pattern

Code

```
# — Cell 5: Evaluator-Optimizer Pattern —————
class EvaluatorOptimizerWorkflow:
    """ Implements the iterative Evaluator-Optimizer refinement cycle. Demonstrates automated code generation with quality evaluation and targeted refinement until stopping criteria are met. """
    EVALUATION_CRITERIA = """ Evaluate the provided Python function against ALL of the following criteria:
    1. CORRECTNESS: Does it solve the stated problem
```

```

without logical errors?      2. STYLE: Does it follow PEP 8 style guidelines?      3.
DOCSTRING: Does it include a Google-style docstring?      4. TYPE HINTS: Are all
parameters and return values type-annotated?      5. ERROR HANDLING: Does it handle
edge cases and raise appropriate exceptions?      6. EFFICIENCY: Is the algorithm
reasonably efficient for the problem?      Respond ONLY with this exact JSON
structure:      {      "overall_score": 1-10,      "passed": true/false,
"criteria_results": {      "correctness":      {"passed": true/false,
"feedback": "..."},      "style":      {"passed": true/false,
"feedback": "..."},      "docstring":      {"passed": true/false,
"feedback": "..."},      "type_hints":      {"passed": true/false,
"feedback": "..."},      "error_handling":      {"passed": true/false,
"feedback": "..."},      "efficiency":      {"passed": true/false,
"feedback": "..."}      },      "actionable_improvements": ["specific
improvement 1", "specific improvement 2"],      "passing_threshold_met":
true/false      }      Note: 'passed' is true when overall_score >= 8 AND all
critical criteria pass.      """      def __init__(self, max_iterations: int = 4,
passing_score: int = 8):      self.max_iterations = max_iterations
self.passing_score = passing_score      self.optimizer = BaseAgent(
name="Code Optimizer",      persona=(      "You are a senior
Python software engineer. "      "Write clean, production-quality Python
functions. "      "When provided with evaluative feedback, carefully
address "      "each specific point raised before producing your revised
output. "      "Return ONLY the Python code – no explanatory prose, no
markdown fences."      ),      temperature=0.3,      )
self.evaluator = BaseAgent(      name="Code Evaluator",
persona=self.EVALUATION_CRITERIA,      temperature=0.1,      )      def
_parse_evaluation(self, raw: str) -> Dict[str, Any]:      """Parse and validate
the evaluator's JSON response."""      try:      clean = raw
return json.loads(clean)      except json.JSONDecodeError:      # Return
a default 'needs improvement' result on parse failure      return {
"overall_score": 5,      "passed": False,
"actionable_improvements": ["Evaluation parsing failed; please review manually."],
"passing_threshold_met": False,      }      def _format_feedback(self,
evaluation: Dict[str, Any]) -> str:      """Format evaluation results into a
clear feedback prompt."""      lines = [      f"EVALUATION SCORE:
{evaluation.get('overall_score', 'N/A')}/10",      f"STATUS: {'PASSED' if
evaluation.get('passed') else 'NEEDS IMPROVEMENT'}",      "\nSPECIFIC
IMPROVEMENTS REQUIRED:",      ]      for i, improvement in
enumerate(evaluation.get("actionable_improvements", []), 1):
lines.append(f"  {i}. {improvement}")      criteria =
evaluation.get("criteria_results", {})      if criteria:
lines.append("\nCRITERIA DETAIL:")      for criterion, result in
criteria.items():      status = "✓" if result.get("passed") else "X"
lines.append(f"  {status} {criterion.upper()}: {result.get('feedback', '')}")
return "\n".join(lines)      def execute(self, task_description: str) -> Dict[str,
Any]:      """      Runs the Evaluator-Optimizer cycle until the output
passes      evaluation criteria or the maximum iteration limit is reached.
"""      print(f"\n{'='*60}")      print(f"  EVALUATOR-OPTIMIZER WORKFLOW")
print(f"  Task: {task_description[:55]}...")      print(f"  Max Iterations:
{self.max_iterations} | Passing Score: {self.passing_score}/10")
print(f"{'T='*60}")      current_output = None      evaluation_history = []
feedback = None      for iteration in range(1, self.max_iterations + 1):
print(f"\n— Iteration {iteration}/{self.max_iterations} —")
# — Generation / Refinement Step —      if
iteration == 1:      prompt = f"Write a Python function for the
following task:\n{task_description}"      else:      prompt = (
f"Revise your previous Python function based on this feedback:\n"
f"{feedback}\n\n"
f"Original task:\n{task_description}\n\n"
f"Previous code:\n{current_output}"      )      current_output =
self.optimizer.run(prompt)      print(f"  [Optimizer] Generated
{len(current_output.split(chr(10)))} lines of code.")      # — Evaluation
Step —      eval_prompt = (
f"Task specification:\n{task_description}\n\n"
f"Code to

```

```

evaluate:\n{current_output}"          )          raw_eval      =
self.evaluator.run(eval_prompt)          evaluation      =
self._parse_evaluation(raw_eval)          score          =
evaluation.get("overall_score", 0)          passed          =
evaluation.get("passing_threshold_met", False)
evaluation_history.append({              "iteration": iteration,
"score":      score,                      "passed":      passed,              })
print(f"  [Evaluator] Score: {score}/10 | Passed: {passed}")          # —
Stopping Condition Check —————          if passed or
score >= self.passing_score:              print(f"\n  ✓ Stopping condition met
at iteration {iteration}.")              break              # Prepare feedback
for the next iteration          feedback = self._format_feedback(evaluation)
print(f"  Actionable improvements identified: "
f"{len(evaluation.get('actionable_improvements', []))}")          if iteration
== self.max_iterations:              print(f"\n  ⚠ Maximum iterations reached.
Returning best available output.")              return {              "final_output":
current_output,              "total_iterations":      len(evaluation_history),
"final_score":      evaluation_history[-1]["score"] if evaluation_history else
0,              "passed":      evaluation_history[-1]["passed"] if
evaluation_history else False,              "iteration_scores":      [e["score"] for e
in evaluation_history],              }          # — Execution
—————          eo_workflow =
EvaluatorOptimizerWorkflow(max_iterations=3, passing_score=8) result =
eo_workflow.execute(          "A function that accepts a list of integers and returns a
dictionary "          "containing the mean, median, mode, and standard deviation of the
list, "          "handling empty lists and non-integer inputs gracefully." )
print(f"\n{'='*60}") print(f"  FINAL RESULT SUMMARY") print(f"\n{'='*60}") print(f"
Final Score          : {result['final_score']}/10") print(f"  Total Iterations:
{result['total_iterations']}") print(f"  Score Progression: {' → '.join(map(str,
result['iteration_scores']))}") print(f"  Passed          : {result['passed']}")
print(f"\n  Final Code:") print(result["final_output"])

```

Explanation

This cell provides a sophisticated implementation of the Evaluator-Optimizer pattern applied to automated code generation — a high-value use case where output quality is objectively measurable against concrete criteria.

- EVALUATION_CRITERIA as a Class Constant:** The evaluation rubric is defined as a class-level string constant rather than being embedded in a method or instance variable. This design decision reflects its nature as a stable, shared specification — it defines the quality standards for the entire class of tasks this workflow handles. Modifying the criteria requires changing only this one location, and the criteria are visible at the top of the class definition for immediate comprehension.
- Iteration State Machine:** The execute method implements a controlled iteration loop with distinct behaviour for the first iteration (initial generation from task description) and subsequent iterations (refinement guided by specific feedback). This distinction is critical — blind regeneration without feedback would be statistically unlikely to improve; targeted refinement guided by precise critique systematically drives the output toward the criteria.
- _format_feedback Method:** Rather than passing the raw evaluation JSON to the Optimizer, this method reformats it into a human-readable structured feedback prompt.

This translation is important: LLMs respond more reliably to clearly formatted natural language instructions than to raw JSON data. The formatted feedback specifies both the overall score and granular per-criterion assessments, enabling the Optimizer to prioritise its revisions effectively.

- **Dual Stopping Conditions:** The loop terminates on two conditions: the evaluator's 'passing_threshold_met' flag (reflecting the LLM's holistic quality assessment) OR the score exceeding the passing_score parameter. The OR logic is intentional — a high numeric score should terminate the loop even if the LLM didn't explicitly set the boolean flag, guarding against LLM inconsistency in setting that field.
- **iteration_scores Tracking:** The evaluation history tracks scores across all iterations and is returned in the result dictionary. Visualising this progression (e.g., [5, 7, 9]) provides direct empirical evidence of the pattern's effectiveness and is invaluable for debugging workflows where quality fails to improve across iterations.

Notebook Cell 6: Implementing the Orchestrator-Workers Pattern

Code

```
# — Cell 6: Orchestrator-Workers Pattern —————
class OrchestratorWorkersWorkflow: """ Implements the dynamic Orchestrator-
Workers pattern. The Orchestrator analyses a complex request at runtime,
dynamically plans required sub-tasks, delegates to specialist Workers, and
synthesises a unified final output. Demonstrates: Market Research Report
generation workflow. """ # Registry of available specialist workers
AVAILABLE_WORKERS = {
    "market_analyst": {
        "description": "Analyses market size, trends, and growth projections.",
        "persona": (
            "You are a senior market analyst with expertise in technology "
            "sector research. Provide data-driven analysis with specific "
            "metrics, market size estimates, and growth projections. "
            "Structure your analysis with clear section headings."
        ),
    },
    "competitive_analyst": {
        "description": "Researches competitor landscape and strategic positioning.",
        "persona": (
            "You are a competitive intelligence specialist. Identify key "
            "market players, their value propositions, market shares, "
            "and strategic differentiators. Provide actionable insights."
        ),
    },
    "risk_analyst": {
        "description": "Identifies and assesses risks and regulatory considerations.",
        "persona": (
            "You are a risk and regulatory affairs analyst. Identify "
            "technical, market, regulatory, and operational risks. "
            "Assess each risk's probability and potential impact. "
            "Suggest mitigation strategies."
        ),
    },
    "opportunity_analyst": {
        "description": "Identifies strategic opportunities and use cases.",
        "persona": (
            "You are a strategic opportunities analyst. Identify "
            "specific, "
            "actionable market opportunities, niche segments, and "
            "innovative "
            "use cases. Prioritise opportunities by potential "
            "value and feasibility."
        ),
    },
    "synthesiser": {
        "description": "Integrates multiple research inputs into a cohesive report.",
        "persona": (
            "You are an expert report writer and strategic "
            "consultant. "
            "Synthesise the provided research sections into a "
            "coherent, "
            "well-structured executive report. Ensure logical flow,
```

```

eliminate " "redundancy, and highlight key strategic implications.
" "Format the output as a professional market research report."
), }, } def __init__(self): # Instantiate all worker
agents from the registry self.workers = { worker_id: BaseAgent(
name=worker_id.replace("_", " ").title(),
persona=config["persona"], temperature=0.6,
max_tokens=1200, ) for worker_id, config in
self.AVAILABLE_WORKERS.items() } # The Orchestrator: the planning
and coordination brain self.orchestrator = BaseAgent(
name="Orchestrator", persona=( "You are a strategic
research project manager. " "When given a research request, analyse
it and create an " "optimal execution plan. "
"Available worker IDs: market_analyst, competitive_analyst, "
"risk_analyst, opportunity_analyst. " "(Note: 'synthesiser' is
always applied last and need not be planned.) " "Respond ONLY with
a JSON array of subtask objects: " "[{
'worker_id': '<id>', 'task_description': '<specific task for
this worker>', 'priority': 1-5' }] "
"Order by priority (1=highest). Include only workers relevant to the request."
), temperature=0.3, ) def _plan_subtasks(self,
research_request: str) -> List[Dict[str, Any]]: """Orchestrator plans the
sub-tasks dynamically at runtime.""" raw = self.orchestrator.run(
f"Plan the research sub-tasks for this request:\n{research_request}" )
try: clean = raw subtasks = json.loads(clean) #
Validate all planned workers exist in registry valid_subtasks = [
s for s in subtasks if s.get("worker_id") in self.AVAILABLE_WORKERS
and s.get("worker_id") != "synthesiser" ] return
sorted(valid_subtasks, key=lambda x: x.get("priority", 99)) except
(json.JSONDecodeError, TypeError): # Fallback: use all available
research workers print(" ⚠️ Orchestration planning parse error; using
default plan.") return [ {"worker_id":
"market_analyst", "task_description": research_request, "priority": 1},
{"worker_id": "competitive_analyst", "task_description": research_request,
"priority": 2}, {"worker_id": "opportunity_analyst",
"task_description": research_request, "priority": 3}, ] def
_execute_subtasks( self, subtasks: List[Dict[str, Any]],
research_request: str, ) -> Dict[str, str]: """Execute all planned sub-
tasks; returns worker_id → output mapping.""" results = {} for
subtask in subtasks: worker_id = subtask["worker_id"]
task_desc = subtask["task_description"] worker =
self.workers[worker_id] print(f" [{worker.name}] Executing:
{task_desc[:60]}...") results[worker_id] = worker.run(
task_desc, context=f"Overall research request: {research_request}"
) print(f" [{worker.name}] ✓ Complete.") return results
def _synthesise( self, worker_outputs: Dict[str, str],
original_request: str, ) -> str: """Orchestrator's synthesis phase:
integrates all worker outputs.""" sections = "\n\n".join([
f"=== {wid.replace('_', ' ').upper()} ===\n{output}" for wid, output in
worker_outputs.items() ]) return self.workers["synthesiser"].run(
f"Synthesise the following research sections into a comprehensive report:",
context=( f"Original Request: {original_request}\n\n"
f"Research Sections:\n{sections}" ) ) def execute(self,
research_request: str) -> Dict[str, Any]: """ Executes the full
Orchestrator-Workers workflow: Plan → Delegate → Execute → Synthesise.
""" print(f"\n{'='*60}") print(f" ORCHESTRATOR-WORKERS WORKFLOW")
print(f" Request: {research_request[:55]}...") print(f"{'='*60}")
# Phase 1: Dynamic Planning print("\n[Phase 1] Orchestrator planning sub-
tasks...") subtasks = self._plan_subtasks(research_request)
print(f" Planned {len(subtasks)} sub-tasks:") for t in subtasks:
worker_name = t['worker_id'].replace('_', ' ').title() print(f"
Priority {t.get('priority', '?')}: {worker_name}") # Phase 2: Parallel
Execution (sequential here for simplicity) print("\n[Phase 2] Executing
worker sub-tasks...") worker_outputs = self._execute_subtasks(subtasks,

```



```

research_request)          # Phase 3: Synthesis          print("\n[Phase 3]
Synthesiser integrating outputs...")          final_report =
self._synthesise(worker_outputs, research_request)          print("  ✓ Synthesis
complete.")          return {          "research_request": research_request,
"subtasks_executed": len(subtasks),          "workers_used":
[t["worker_id"] for t in subtasks],          "final_report":          final_report,
}          # — Execution

OrchestratorWorkersWorkflow() result = orch_workflow.execute(          orch_workflow =
comprehensive market research analysis on the adoption of "          "Conduct a
Models in enterprise software development, covering "          "Large Language
competitive landscape, opportunities, and risks." )          print(f"\n{'='*60}")          print(f"
FINAL MARKET RESEARCH REPORT")          print(f"  Workers Used: {'',
'.join(result['workers_used'])}")          print(f"  Sub-tasks Executed:
{result['subtasks_executed']}")          print(f"{'='*60}")
print(result["final_report"][:1200] + "... \n[Report continues...]")

```

Explanation

This cell implements the most architecturally sophisticated pattern — the Orchestrator-Workers workflow — applied to a market research report generation system. Every design decision reflects the core principle of dynamic, runtime orchestration.

- **AVAILABLE_WORKERS Registry:** The class-level AVAILABLE_WORKERS dictionary serves as a formal capability registry — a declarative manifest of all specialist agents available in the system, along with a human-readable description of each one's function. This description is not merely documentation — it is included in the Orchestrator's persona so that the Orchestrator has accurate knowledge of the available workforce when formulating its plan.
- **Worker Instantiation via Comprehension:** All worker agents are instantiated programmatically from the registry using a dictionary comprehension. This pattern means that adding a new specialist agent requires only adding an entry to AVAILABLE_WORKERS — the instantiation loop handles the rest automatically. This represents the extensibility benefit of the registry pattern.
- **The Orchestrator's Constrained Output Format:** The Orchestrator persona specifies that it must respond exclusively with a JSON array of subtask objects with defined fields. This structural constraint transforms the Orchestrator's natural language reasoning into machine-parseable planning output — the essential bridge between LLM intelligence and programmatic execution.
- **_plan_subtasks — Runtime Dynamic Planning:** This method embodies the defining characteristic of the Orchestrator-Workers pattern: the sub-task plan is generated at runtime based on the specific incoming request, not retrieved from a fixed template. For a market research request, the Orchestrator might plan market analysis, competitive analysis, and opportunity analysis. For a different request (e.g., technical feasibility

assessment), it would generate an entirely different plan — all without any code changes.

- **Synthesiser as a Specialised Worker:** The synthesiser worker is excluded from the Orchestrator's planning scope (note: 'synthesiser is always applied last and need not be planned') and is always invoked as the final step. This architectural decision reflects the synthesiser's role as an invariant final stage rather than a variable research component — it is part of the workflow's structure, not its dynamic content.
- **Sections Formatted for Synthesis:** In `_synthesise`, worker outputs are concatenated with clear section headers before being passed to the synthesiser. This formatting serves a critical purpose: it preserves the attribution of each section to its source worker, enabling the synthesiser to understand the provenance of each input and produce a report that properly integrates diverse perspectives rather than merely concatenating them.

Notebook Cell 7: System Integration — Combining Multiple Patterns

Code

```
# — Cell 7: Integrated Pattern Demonstration ————— #
This cell demonstrates how multiple patterns can be composed into # a single,
cohesive production-grade agentic pipeline. class IntegratedAgenticPipeline:
"""    An integrated pipeline combining Routing → Prompt Chaining → Evaluation.
Demonstrates pattern composition for a content generation system that:    1.
Routes incoming requests to appropriate specialist workflows.    2. Uses prompt
chaining to produce structured content.    3. Applies an evaluation step for
quality assurance.    """    def __init__(self):        # High-level router:
determines which specialised chain to invoke        self.router =
RoutingWorkflow()        # Content quality evaluator        self.evaluator =
BaseAgent(            name="Pipeline Evaluator",            persona=(
"You are a senior content quality evaluator. "            "Assess the provided
content against: "            "(1) Relevance to the original query, "
"(2) Accuracy and factual soundness, "            "(3) Clarity and
readability, "            "(4) Actionability – does it give the user clear
next steps? "            "Respond ONLY with JSON: "            '{"score":
1-10, "approved": true/false, "improvement_notes": "..."}'            ),
temperature=0.1,        )        def process(self, user_query: str) -> Dict[str,
Any]:            """            Full pipeline: Route → Generate → Evaluate → Return.
"""            print(f"\n{'—'*60}")            print(f"    INTEGRATED PIPELINE
PROCESSING")            print(f"    Query: {user_query[:55]}...")
print(f"{'—'*60}")            # — Stage 1: Route the query
            print("\n[Stage 1] Routing...")
routed = self.router.handle(user_query)            initial_response =
routed["response"]            category            = routed["category"]            # —
Stage 2: Quality Evaluation            print("\n[Stage
2] Quality evaluation...")            eval_prompt = (                f"Original query:
{user_query}\n\n"                f"Generated response: {initial_response}"            )
raw_eval    = self.evaluator.run(eval_prompt)            clean_eval = raw_eval
try:                evaluation = json.loads(clean_eval)            except
```

```

json.JSONDecodeError:          evaluation = {"score": 7, "approved": True,
"improvement_notes": "Eval parse error."}          score =
evaluation.get("score", 0)          approved = evaluation.get("approved", True)
print(f"  Quality Score: {score}/10 | Approved: {approved}")          # — Stage 3:
Refinement if needed —————          final_response =
initial_response          refined          = False          if not approved and score <
7:          print("\n[Stage 3] Quality below threshold – refining response...")
refiner = BaseAgent(          name="Response Refiner",
persona=(          "You are an expert response refiner. Given an
original response "          "and specific improvement notes, produce an
enhanced version that "          "addresses all identified shortcomings
while preserving strengths."          ),          temperature=0.4,
)          final_response = refiner.run(          f"Original query:
{user_query}\n\n"          f"Original response: {initial_response}",
context=f"Improvement notes: {evaluation.get('improvement_notes', '')}"
)          refined = True          print("  ✓ Refinement complete.")
return {          "query":          user_query,          "category":
category,          "initial_score":          score,          "refined":
refined,          "final_response":          final_response,          }          # —
Demonstration —————
pipeline = IntegratedAgenticPipeline() demo_query = (          "I'm trying to integrate
your API into my Python application, "          "but I keep getting a 401 Unauthorized
error despite setting my API key correctly. "          "Can you help me debug this?" )
pipeline_result = pipeline.process(demo_query) print(f"\n{'—'*60}") print(f"
PIPELINE SUMMARY") print(f"{'—'*60}") print(f"  Category Routed :
{pipeline_result['category']}") print(f"  Quality Score :
{pipeline_result['initial_score']}/10") print(f"  Response Refined :
{pipeline_result['refined']}") print(f"\n  Final Response:")
print(pipeline_result["final_response"])

```

Explanation

This final cell demonstrates a critically important architectural principle: agentic patterns are composable building blocks, not mutually exclusive design choices. Production-grade systems frequently combine multiple patterns into integrated pipelines that leverage the strengths of each.

- **Pattern Composition:** The `IntegratedAgenticPipeline` class orchestrates three patterns in sequence: Routing (to dispatch to the appropriate specialist), followed by inline quality Evaluation (to assess response quality), with conditional Refinement (a simplified Evaluator-Optimizer cycle triggered only when quality falls below threshold). This composition is efficient: specialisation, quality assurance, and refinement all operate within a single cohesive pipeline.
- **Lazy Refinement Pattern:** The refinement stage is conditionally triggered only when `approved` is `False` and `score < 7`. This 'lazy refinement' approach is architecturally sound for production: it avoids the computational cost of refinement for responses that already meet quality standards, reserving additional processing for cases where it adds genuine value.
- **Inline Agent Instantiation:** The refiner `BaseAgent` is instantiated within the `process` method rather than in `__init__`. This is an intentional design choice for infrequently used agents: it avoids the (minor) overhead of instantiation when refinement is not needed. In

practice, for frequently accessed agents, `__init__` instantiation is preferable to avoid repeated construction overhead.

- **Structured Pipeline Result:** The return dictionary captures the full provenance of the result: the routing category, initial quality score, whether refinement was triggered, and the final response. This metadata is invaluable for production monitoring — it enables analysis of routing accuracy, average quality scores by category, refinement rates, and system performance trends over time.

Chapter Summary

This chapter has established the complete theoretical and practical foundation for agentic AI workflows. The following table summarises the core patterns covered and their most appropriate application contexts:

Pattern	Core Mechanism	Best Applied When	Key Trade-off
Prompt Chaining	Sequential LLM calls with output-to-input passing	Task can be cleanly decomposed into sequential sub-tasks	Error propagation risk requires validation at each gate
Routing	Classification → conditional dispatch	Input diversity requires different specialist handlers	Misclassification sends tasks to wrong specialists
Parallelisation	Concurrent independent sub-task execution	Task has independent components; latency matters	Sub-tasks must be independent; aggregation adds complexity
Evaluator-Optimizer	Iterative generate-critique-refine cycle	High quality standards; objective criteria measurable	Iteration cost; requires well-defined stopping conditions
Orchestrator-Workers	Dynamic runtime planning + specialist delegation	Problem complexity is high; solution path unpredictable	Orchestrator plan quality determines overall system quality

Next Steps

Having mastered these foundational patterns, the natural progression is to explore:

- **Memory Systems:** Short-term, long-term, and episodic memory architectures for stateful agents.
- **Tool Integration:** Building robust tool libraries with structured error handling.
- **Multi-Agent Frameworks:** LangGraph, CrewAI, and AutoGen for production-grade orchestration.
- **Evaluation & Observability:** LLM evaluation frameworks (RAGAS, TruLens) and tracing with LangSmith.

— End of Chapter —